

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Reverse Engineering

Code of Conduct:

I M. MUKKARAM (192524357) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

M. MUKKARAM

Annexure A

Reverse Engineering

1. Reverse Engineering a Recommendation System

A large-scale e-commerce platform provides highly accurate product recommendations using real-time data. Analyze the system and reverse engineer its architecture using PySpark and RDD operations. Identify how data is collected, processed, and transformed, and infer the role of intelligent agents in updating recommendations dynamically.

1. Reverse Engineering a Recommendation System

A large-scale e-commerce recommendation system collects huge amounts of real-time data such as customer searches, product views, clicks, purchases, ratings, and browsing history. The main objective of the system is to analyze this data and recommend relevant products to users.

Data Collection:

User interaction data is collected from websites, mobile applications, product databases, and transaction systems. The collected data may include user ID, product ID, category, price, rating, search history, and purchase history.

Data Processing using PySpark:

PySpark is used to process the large volume of data in a distributed manner. The data can be loaded into RDDs, which divide the data into partitions and allow multiple machines to process it simultaneously.

RDD Operations:

Transformations such as `map()`, `filter()`, `flatMap()`, and `reduceByKey()` can be used to clean and organize the data. For example, `filter()` can remove invalid records, while `reduceByKey()` can calculate the number of interactions for each product.

Recommendation Generation:

The processed information is used to identify products that are relevant to each customer. Products can be ranked according to previous purchases, browsing behavior, ratings, and similar users' interests.

Role of Intelligent Agents:

Intelligent agents continuously monitor customer activity and recommendation performance. When a user searches for or purchases a product, the agent can update the user's preference information and trigger a new recommendation process.

Conclusion:

The system combines PySpark and RDD-based distributed processing with intelligent agents to provide fast and personalized recommendations. This architecture allows the e-commerce platform to handle large-scale data while dynamically updating recommendations.

2. Reverse Engineering a Smart Traffic Management System

A city-wide traffic system adapts signals in real time based on vehicle density and congestion patterns. Reverse engineer the underlying AI system by describing the data pipeline using RDD-based streaming, identifying agent interactions, and explaining how decisions are made under time constraints.

2. Reverse Engineering a Smart Traffic Management System

A smart traffic management system uses real-time information to control traffic signals according to vehicle density and congestion. The objective is to reduce waiting time, traffic congestion, and unnecessary fuel consumption.

Data Collection:

Traffic information can be collected from cameras, sensors, GPS devices, traffic signals, and connected vehicles. Important information includes vehicle count, vehicle speed, road occupancy, and congestion level.

RDD-Based Data Pipeline:

The collected streaming data can be distributed across a PySpark cluster using RDDs. Each RDD contains a portion of the incoming traffic data, allowing different machines to process data simultaneously.

RDD Operations:

Operations such as `map()`, `filter()`, and `reduceByKey()` can process traffic information. The `map()` operation can convert raw sensor readings into useful values, while `filter()` can remove incorrect readings. `reduceByKey()` can calculate the total number of vehicles at each road junction.

Intelligent Agent Interaction:

Intelligent agents can represent individual traffic intersections. Each agent monitors the traffic condition at its location and communicates relevant information with nearby agents.

Decision Making:

If one road has heavy traffic, the corresponding agent can request a longer green-light duration. Decisions are made using current traffic density and congestion information so that signals can respond quickly.

Conclusion:

The system combines real-time data processing, PySpark RDD operations, and intelligent agents to dynamically control traffic signals. This distributed approach allows the system to respond quickly to changing traffic conditions.

3. Reverse Engineering a Fraud Detection Pipeline

A banking system flags fraudulent transactions within milliseconds. By observing its outputs and behavior, reconstruct the distributed AI pipeline using PySpark RDDs. Explain how feature extraction, anomaly detection, and agent-based alert mechanisms are likely implemented.

3. Reverse Engineering a Fraud Detection Pipeline

A banking fraud detection system analyzes financial transactions and identifies suspicious activities within a very short time. Since banks process a large number of transactions, distributed processing is required for fast and reliable detection.

Data Collection:

The system collects transaction details such as transaction amount, account number, location, time, payment method, and transaction frequency. Historical transaction records can also be used to identify normal customer behavior.

PySpark and RDD Processing:

PySpark can distribute transaction data across multiple computing nodes. The transaction records are stored in RDDs and processed in parallel, allowing large numbers of transactions to be analyzed efficiently.

Feature Extraction:

Useful features are extracted from the transaction data. Examples include unusually large transaction amounts, frequent transactions within a short period, unusual locations, and changes from normal spending patterns.

Anomaly Detection:

The extracted features are analyzed to determine whether a transaction is normal or suspicious. Transactions that significantly differ from expected behavior can be assigned a higher fraud risk score.

RDD Operations:

map() can be used to extract transaction features, filter() can identify transactions satisfying suspicious conditions, and reduceByKey() can aggregate transaction information by account or location.

Agent-Based Alert Mechanism:

Intelligent agents monitor transaction results and respond when suspicious activity is detected. An alert agent can notify the banking system, while another agent can monitor the customer's transaction history for additional evidence.

Conclusion:

The fraud detection pipeline uses PySpark RDDs for scalable transaction processing, feature extraction for identifying useful patterns, anomaly detection for recognizing suspicious behavior, and intelligent agents for rapid alert generation.

4. Reverse Engineering a Personalized Learning System

An online learning platform adapts content difficulty based on student performance and engagement. Reverse engineer the system design by identifying how learner data is processed using RDD transformations, and explain how intelligent agents contribute to adaptive learning and feedback mechanisms.

Reverse Engineering a Personalized Learning System

A personalized learning system automatically adapts educational content according to a student's performance, learning progress, and engagement. The main purpose is to provide suitable learning materials and feedback for each student.

Data Collection:

The system collects information such as quiz scores, assignment marks, time spent on lessons, completed topics, number of attempts, and student interaction with learning materials.

Processing using PySpark:

Large amounts of learner data can be processed using PySpark. Student records are distributed into RDD partitions so that multiple computers can process the data simultaneously.

RDD Transformations:

The `map()` operation can transform student records into performance measures. The `filter()` operation can identify students or activities that satisfy particular conditions. `reduceByKey()` can calculate average scores or aggregate performance for individual students.

Learning Analysis:

The system analyzes the processed data to identify the student's strengths and areas requiring improvement. For example, consistently low scores in a particular topic may indicate that additional learning material is required.

Role of Intelligent Agents:

Intelligent agents monitor student progress and select suitable learning resources. A recommendation agent can suggest additional exercises, while a feedback agent can provide guidance based on student performance.

Adaptive Learning:

When a student's performance improves, the system can provide more advanced content. If performance decreases, the system can recommend revision materials or simpler exercises.

Conclusion:

The personalized learning system combines PySpark, RDD transformations, and intelligent agents to analyze learner data and provide adaptive educational content. This approach improves personalization and allows the learning platform to respond dynamically to student needs.

5. Reverse Engineering a Cloud-Based AI Monitoring System

A cloud platform automatically scales resources and detects system anomalies in real time. Reverse engineer the architecture by analyzing how PySpark processes large-scale logs, how RDD operations support fault tolerance, and how intelligent agents coordinate monitoring, scaling, and anomaly response.

5. Reverse Engineering a Cloud-Based AI Monitoring System

A cloud-based AI monitoring system continuously observes cloud resources, application logs, and system performance. It automatically identifies abnormal conditions and can scale resources according to workload requirements.

Data Collection:

The system collects large amounts of information from cloud servers, applications, databases, network devices, and monitoring services. Examples include CPU usage, memory usage, network traffic, application errors, and server response time.

PySpark Log Processing:

PySpark can process large-scale log data in a distributed environment. The logs can be stored in RDDs and divided into partitions, allowing multiple computing nodes to process different portions of the data simultaneously.

RDD Operations:

The `map()` operation can extract useful information from log records. The `filter()` operation can identify error messages or abnormal events. `reduceByKey()` can aggregate errors, resource usage, or events by server or application.

Fault Tolerance:

RDDs support fault-tolerant distributed processing because lost partitions can be recomputed from their previous transformations. This helps the monitoring system continue processing even if a processing node fails.

Intelligent Agent Coordination:

Different intelligent agents can perform different monitoring tasks. A monitoring agent observes system conditions, an anomaly detection agent identifies unusual behavior, and a scaling agent manages resource requirements.

Scaling and Anomaly Response:

When workload increases, the scaling agent can recommend or initiate additional resources. When abnormal system behavior is detected, the response agent can generate alerts and initiate appropriate recovery actions.

Conclusion:

The cloud monitoring system combines PySpark and RDD operations for large-scale log processing and fault-tolerant computation. Intelligent agents coordinate monitoring, anomaly detection, scaling, and response activities, making the cloud environment more reliable and responsive.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation